

Google Ads API - Basic Access Application: Tool Design Document

Applicant / Developer: <Your Name>
Contact email: your-email@example.com
Company website: https://your-website.example.com/
Manager (MCC) account ID: 123-456-7890
Google Cloud project ID: your-gcp-project-id
Company type: Advertiser (self-use)
Date: 2026-07-11

1. Product Overview

This tool is a single-user, self-hosted automation server that lets the developer manage their own Google Ads account through natural-language commands in Claude Code (an AI coding assistant). It is not a product sold to third parties and is not used to manage other advertisers' accounts.

Concretely, it is a small HTTP service ("MCP server") that exposes a fixed set of Google Ads operations (reporting and campaign management) as callable tools. The developer types requests like "create a paused search campaign with a 30,000 KRW daily budget," and the server translates that into the corresponding Google Ads API calls against the developer's own account.

Primary purpose: reduce the manual, repetitive clicking involved in building and reporting on the developer's own advertising campaigns (search and Demand Gen), and make that process reproducible across the developer's own computers.

2. Business Model

The developer runs advertising for their own business (see company website above). There is no external customer for this tool and no resale. The tool is internal tooling that improves the efficiency of managing the developer's own ad spend. Revenue is generated by the developer's underlying business, not by the tool itself.

3. Intended Users and Access

- Users: Internal only - the developer (account owner) alone.
- Access control: The server is deployed on Google Cloud Run behind a random bearer token (32-byte, `secrets.token_urlsafes`). Only requests carrying that token are served. The token is stored in Google Secret Manager and shared with no one.
- No public sign-up, no multi-tenant use, no client accounts. The OAuth credential belongs to the developer's own Google account and authorizes access only to accounts that account can reach.

4. Google Ads API Usage

The tool uses a fixed, auditable set of operations. It does not perform bulk data harvesting, does not access accounts outside the developer's own manager account, and does not implement any prohibited functionality.

Reporting (read):

- `GoogleAdsService.SearchStream / Search (GAQL)` - campaign, ad group, keyword and metrics reporting for the developer's own accounts.
- `CustomerService.ListAccessibleCustomers`, `customer_client` queries - list the developer's own accounts under the manager account.

Campaign management (write) - search campaigns:

- `CampaignBudgetService`, `CampaignService`, `AdGroupService`, `AdGroupCriterionService` (keywords / negative keywords), `AdGroupAdService` (Responsive Search Ads), `CampaignCriterionService` (geo / language targeting).

Campaign management (write) - Demand Gen:

- `AssetService` (image / logo / YouTube video assets), Demand Gen campaign, ad group and ad creation, and reading back ad policy-approval status.

All creation defaults to PAUSED, and enabling or deleting requires an explicit confirmation argument, so nothing goes live or is destroyed by accident.

5. System Architecture and Design

```
Claude Code (developer's PC)
| natural-language request -> fixed tool call
| HTTPS, Authorization: Bearer <token>
v
Cloud Run service (FastMCP, Python)      <- this tool
| developer-token + OAuth authorized_user credential
v
Google Ads API -> developer's own Google Ads accounts
```

- Client: Claude Code connects to the server over HTTPS using the Model Context Protocol.
- Server: A Python (FastMCP / Starlette / uvicorn) service on Cloud Run. It exposes a fixed list of tools (sections 4). Each tool maps to specific Google Ads API calls; there is no arbitrary/free-form query execution beyond read-only GAQL that the developer explicitly writes.
- Credentials: The Google Ads developer token, the OAuth `authorized_user` credential (`client_id` / `client_secret` / `refresh_token`), and the bearer token are stored in Google Secret Manager and injected as environment variables at runtime. Nothing is hardcoded.
- Statelessness: The server stores no advertiser data. It reads/writes on demand and returns results to the client. There is no database and no long-term storage of Ads data.

6. Data Handling, Privacy and Security

- No storage of Google Ads data or PII. Responses are returned to the developer's own client session and not persisted server-side.
- Secret redaction: Tool output is passed through a redaction filter that masks any token-like strings before it is returned, so credentials never leak into logs or the chat transcript.
- Transport security: All access is over HTTPS. The `/mcp` endpoint requires the bearer token; only `/` and `/healthz` are open (and return no sensitive data).
- Least exposure: Secrets live only in Secret Manager with `secretAccessor` granted solely to the Cloud Run runtime service account.

7. Policy Compliance

- Required Minimum Functionality: The tool provides campaign management and performance reporting for the developer's own account - a standard, compliant use of the API.
- Safe by design: New campaigns/ad groups/ads are created PAUSED; enabling and deletion require

explicit confirmation flags. This prevents accidental spend or destructive changes.

- No prohibited use: No credential sharing, no unauthorized access to third-party accounts, no scraping beyond the developer's own reporting, no arbitrage or policy-circumventing automation.
- Single account scope: Operations are limited to accounts under the developer's own manager account (123-456-7890).

8. Summary

This is internal, single-user tooling for the developer to manage their own Google Ads advertising more efficiently and reproducibly via Claude Code. It uses a fixed, auditable set of reporting and campaign-management API calls, stores no advertiser data, keeps all secrets in Secret Manager, and is designed with safe defaults (paused creation, explicit confirmation for enable/delete). Basic access is requested to allow these operations against the developer's own production Google Ads account.